

.NET 10 · REACT · DOCKER

ASPOTUS

Архитектура и технологическое устройство системы

Каталог автомобилей и запчастей · Заказы · Резервирование · Уведомления

Команда: Антон Лядов · Максим Сафронов · Алексей Силаев · Дмитрий Шадчнев · Елена Карлштейн

Что решает система и для кого

ПОКУПАТЕЛЬ

Каталог, поиск, карточка товара, корзина, оформление и история заказов

МОДЕРАТОР

Управление брендами, моделями, поколениями, автомобилями и запчастями

ОПЕРАТОР

Работа с реальными заказами и перевод по допустимым статусам

АДМИНИСТРАТОР

Пользователи, роли, каталог, заказы и общие показатели

Как читать архитектурные схемы

Короткий словарь: что означает каждый технический термин и зачем он нужен системе

API GATEWAY

Единая «входная дверь». Проверяет токен и роль, затем отправляет запрос нужному сервису.

СЕРВИС / API

Отдельный модуль для одной бизнес-области: каталог, заказы, файлы или уведомления.

БАЗА ДАННЫХ

Собственная долговременная память сервиса. Общей базы между сервисами нет.

КЭШ / REDIS

Быстрая копия часто читаемых данных. Ускоряет ответы, но не является источником истины.

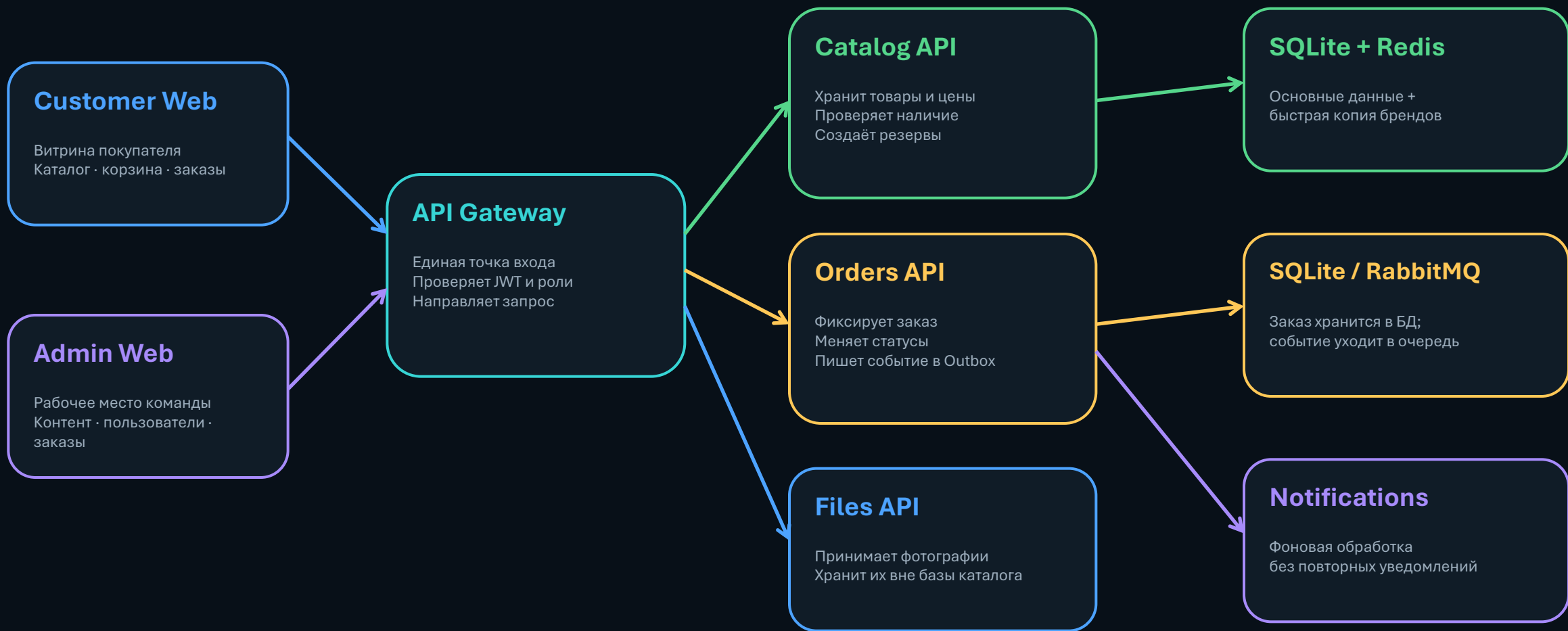
БРОКЕР / RABBITMQ

Очередь доставки событий. Получатель может обработать сообщение позже и повторить попытку.

WORKER

Фоновый процесс без интерфейса: читает очередь, выполняет работу и защищается от дублей.

Карта системы: от браузера до данных



Контейнерная топология Docker Compose

9 контейнеров · одна backend-сеть · постоянные данные вне жизненного цикла контейнеров

customer-web

Сайт покупателя · порт 5173

admin-web

Панель команды · порт 5174

gateway

Общий вход · порт 5230

catalog-api

Товары и резервы · порт 5299

orders-api

Заказы и статусы · порт 5115

files-api

Фотографии · порт 5133

notifications

Фоновая обработка событий

redis

Ускоряющий кэш · порт 6379

rabbitmq

Очередь · порты 5672 / 15672

Volumes: gateway.db · catalog.db · orders.db · notifications.db · RabbitMQ data

Gateway, аутентификация и авторизация



- Gateway централизованно применяет правила доступа к proxied endpoints.
- В downstream передаются X-User-Id, X-User-Email и X-User-Roles.
- Служебный вызов Orders → Catalog защищён X-Internal-API-Key.
- Изменение каталога разрешено ContentModerator/Admin; статусов заказов — Operator/Admin.

Catalog API: владелец товарных данных

Controllers

Принимают HTTP-запросы, проверяют их форму и возвращают коды ответа.



Services

Применяют бизнес-правила: нормализацию, валидацию и проверку наличия.



Repositories

Изолируют EF-запросы и детали чтения или записи данных.



EF Core + SQLite

Сохраняют товары, изображения, остатки и резервы в catalog.db.

Redis cache

Кэш списка и отдельных брендов
TTL по умолчанию: 10 минут
При сбое — работа напрямую с SQLite

Files API

Загрузка изображений в S3-совместимое хранилище
Каталог хранит file key, URL и порядок

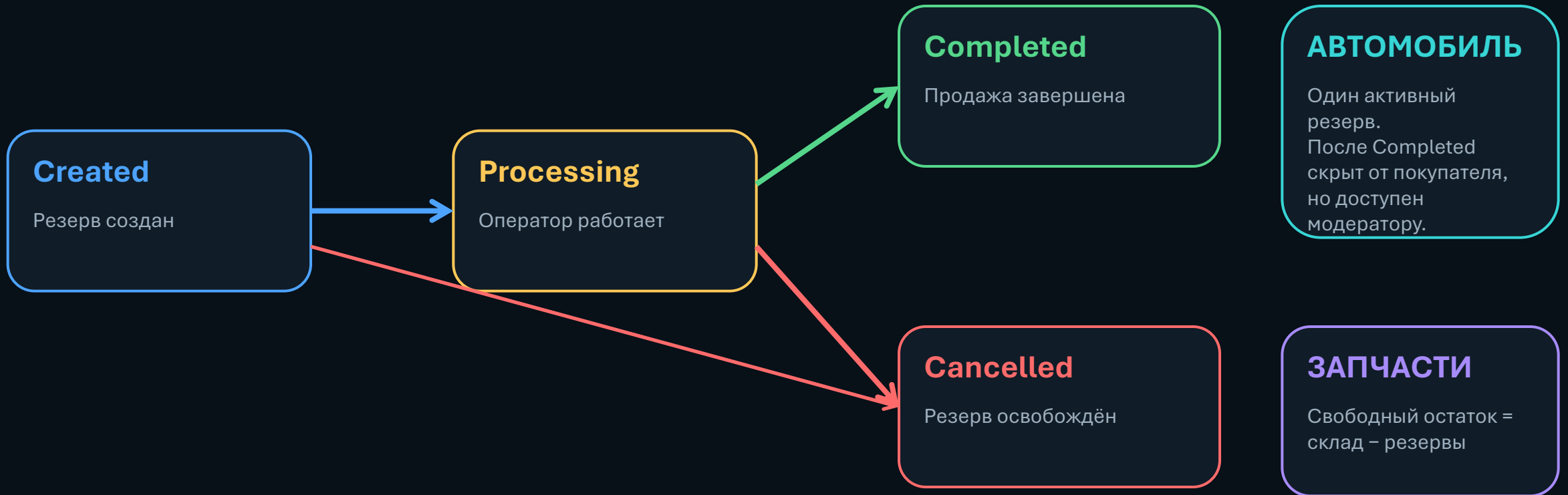
Источник истины: цена + наличие + резерв

Оформление заказа: синхронный путь



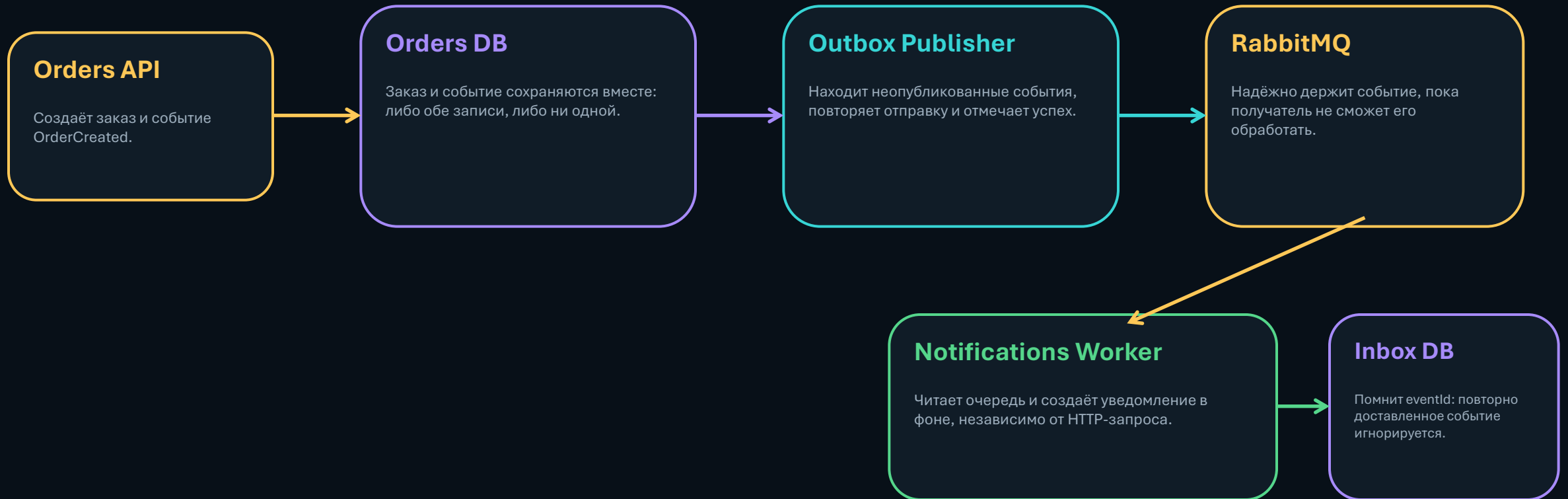
Цена из браузера не считается доверенной: заказ получает стоимость из Catalog API.

Заказ и резервирование: единая модель состояния



Reserve / Release / Complete идемпотентны по orderId

Асинхронные уведомления: Outbox + Inbox



Результат: at-least-once доставка без повторной бизнес-обработки.

Данные разделены по владельцам

gateway.db

Кто пользователь и что ему разрешено: Identity, Users, Roles.

catalog.db

Что продаём, сколько доступно и что зарезервировано.

orders.db

Что заказал клиент, по какой цене и в каком статусе.

notifications.db

Какие события уже обработаны и не должны повторяться.

НЕТ ОБЩЕЙ БАЗЫ ДАННЫХ

- Сервисы общаются через HTTP-контракты и события, а не через таблицы соседа.
- Заказ хранит снимок названия и цены — история не меняется вместе с каталогом.
- SQLite-файлы смонтированы в host volumes и переживают пересоздание контейнеров.

Два интерфейса — разные рабочие сценарии

CUSTOMER WEB

React 19 · Vite 8 · React Bootstrap

Каталог и поиск
Карточка и фотогалерея
Корзина со счётчиком
Checkout и профиль заказов

ADMIN WEB

React 19 · Vite 8 · Material UI

Ролевое меню
CRUD каталога и изображений
Операторская панель на реальных данных
Статусы заказов

Общий принцип: API через Gateway · JWT · lazy routes · адаптивный UI

Надёжность и эксплуатационные свойства

Автомиграции

При старте сервис приводит схему своей БД к ожидаемой версии.

Компенсация

Если заказ не сохранился, созданный резерв освобождается автоматически.

Идемпотентность

Повтор Reserve / Release / Complete не портит состояние и не дублирует результат.

Cache fallback

Если Redis недоступен, Catalog читает SQLite: медленнее, но продолжает работать.

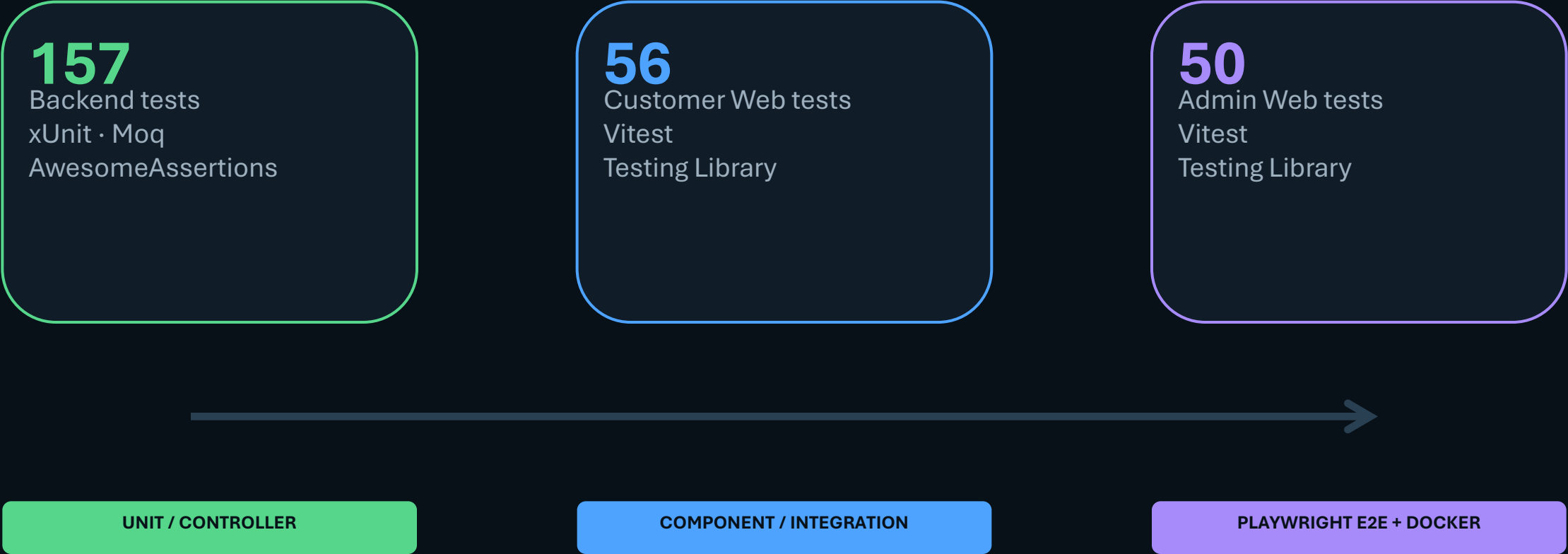
Единые ошибки

Middleware превращает доменные исключения в понятные HTTP-коды для клиентов.

Диагностика

Swagger показывает контракты, логи объясняют сбои, health checks проверяют зависимости.

Тестовая стратегия



E2E проверяет доступность, защиту маршрутов, вход администратора и полный цикл резерва автомобиля.

Технологический стек

BACKEND

.NET 10 и ASP.NET Core реализуют бизнес-API; EF Core работает с SQLite.

EDGE & SECURITY

YARP направляет запросы; Identity, JWT и RBAC защищают операции.

DATA & MESSAGING

Redis ускоряет чтение; RabbitMQ и Outbox / Inbox доставляют события.

FRONTEND

React 19 + Vite 8; Material UI для admin, React Bootstrap для customer.

QUALITY

xUnit и Vitest проверяют код; Playwright проходит реальные сценарии в браузере.

DELIVERY

Dockerfiles собирают образы; Compose запускает систему и подключает volumes.

Как развивать архитектуру дальше

УЖЕ ЕСТЬ

- Изолированные сервисы и базы
- Gateway + RBAC
- Надёжные события Outbox / Inbox
- Docker Compose и автотесты



СЛЕДУЮЩИЙ УРОВЕНЬ

- Secrets manager и ротация ключей
- PostgreSQL и production migrations
- OpenTelemetry: traces, metrics, logs
- Retry / circuit breaker / RabbitMQ DLQ
- CI/CD, API versioning, container orchestration

ИТОГ

Aspotus — система с ясными границами ответственности

Раздельные данные · контролируемый доступ · доверенные цены · резервирование · надёжные события · автоматические проверки

Вопросы?

Главная архитектурная идея

Каждый сервис владеет своей областью и данными, а согласованность достигается явными контрактами и идемпотентными процессами.